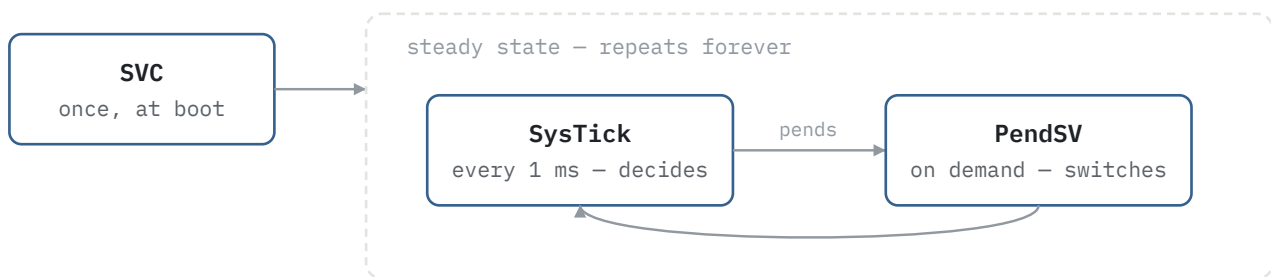


stm32f103 workspace notes

SysTick, PendSV, and SVC

Three exception handlers do all of FreeRTOS's task switching on a Cortex-M3. One decides, one acts, and one only ever runs once. Here's what each of them actually does, in the order they actually do it.

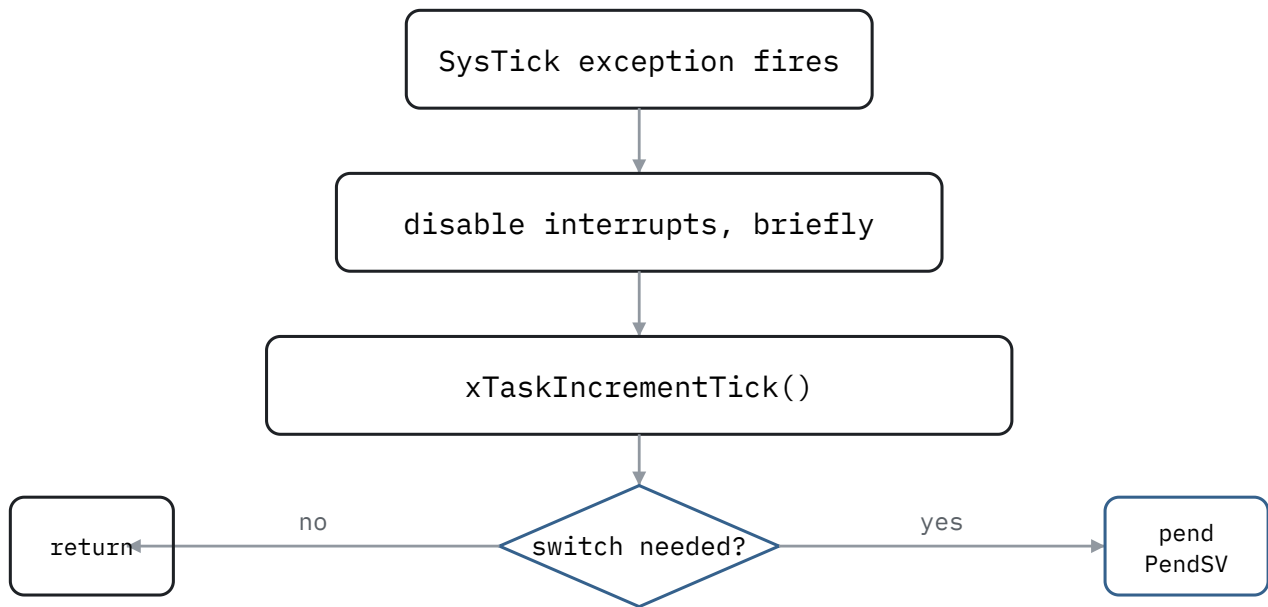
None of this lives in `startup_stm32f103.s`. Your startup file only reserves three slots in the vector table — weak symbols pointing at a default handler, same as every other entry. What actually runs in those slots comes entirely from FreeRTOS's `port.c`, wired in by the three `#define` lines in `FreeRTOSConfig.h`. This page walks through what's on the other end of that wiring.



SysTick_Handler

FIRES EVERY TICK • DECIDES

Every millisecond the SysTick timer expires and this handler runs. Its only job is to ask "did anything change that the scheduler needs to act on?" — a task's `vTaskDelay()` expiring, a time slice running out. It never touches a stack pointer and never decides *which* task runs next. It just flags that a decision needs making, and lets a separate exception carry it out.



FreeRTOS/portable/GCC/ARM_CM3/port.c

```

void xPortSysTickHandler( void )
{
    /* SysTick sits at the lowest priority in the system (see
     * configLIBRARY_LOWEST_INTERRUPT_PRIORITY in FreeRTOSConfig.h) - by
     * the time this runs, nothing more urgent is waiting, so there's
     * nothing to save/restore around the next line. */
    portDISABLE_INTERRUPTS();

    /* bump the tick count; walk the delayed-task list for anyone whose
     * wake time just arrived; returns true if a switch is warranted */
    if( xTaskIncrementTick() != pdFALSE )
    {
        /* don't switch here - just ask PendSV to. one register write. */
        portNVIC_INT_CTRL_REG = portNVIC_PENDSVSET_BIT;
    }

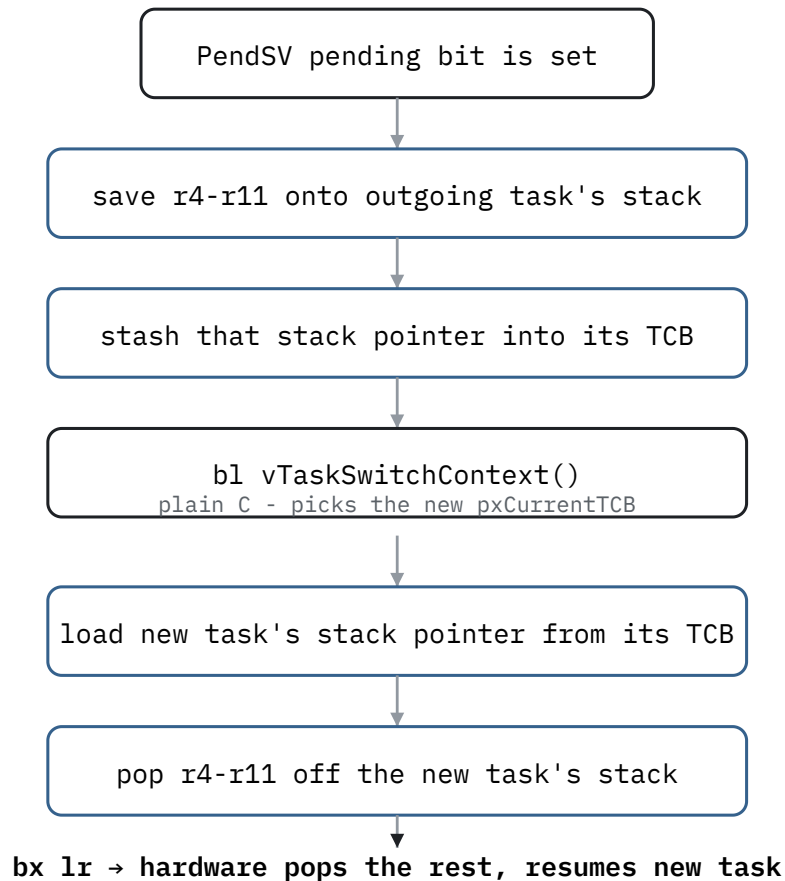
    portENABLE_INTERRUPTS();
}

```

PendSV_Handler

FIRES ON DEMAND • SWITCHES

This is the actual context switch — the eleven instructions that make it look, from every task's point of view, like it was never interrupted. ARM built PendSV specifically for this: it's the one exception you can pend from software, and it's deliberately parked at the lowest priority alongside SysTick, so it only ever runs once every genuinely higher-priority interrupt has finished. That's what stops a switch from ever happening mid-way through something more urgent.



FreeRTOS/portable/GCC/ARM_CM3/port.c – xPortPendSVHandler (naked)

```

mrs    r0, psp                ; r0 = outgoing task's process stack pointer
isb                                ; make sure that read has actually landed

ldr    r3, =pxCurrentTCB      ; r3 = &pxCurrentTCB (the pointer variable itself)
ldr    r2, [r3]               ; r2 = pxCurrentTCB (the outgoing task's TCB)

stmdb  r0!, {r4-r11}          ; push r4-r11 onto THAT task's own stack
str    r0, [r2]               ; TCB->pxTopOfStack = r0 (first member of the struct)

stmdb  sp!, {r3, r14}         ; save r3 and LR on PendSV's own stack, around the C call
mov    r0, #configMAX_SYSCALL_INTERRUPT_PRIORITY
msr    basepri, r0            ; mask anything that's allowed to touch the ready lists

bl     vTaskSwitchContext     ; plain C - updates pxCurrentTCB to the new task
  
```

```

mov    r0, #0
msr    basepri, r0        ; unmask again
ldmia  sp!, {r3, r14}     ; restore r3, LR

ldr    r1, [r3]           ; r1 = pxCurTaskTCB - now the NEW task
ldr    r0, [r1]           ; r0 = new task's saved stack pointer
ldmia  r0!, {r4-r11}      ; pop r4-r11 back off ITS stack

msr    psp, r0            ; PSP now points into the new task's stack
isb
bx     r14                ; exception return - hw auto-pops r0-r3/r12/PC/xPSR
                        ;   from the NEW task's stack. it just resumes.

```

The two `msr basepri` lines are worth noticing: PendSV briefly raises the interrupt mask *only* around the call into `vTaskSwitchContext()` — the one moment it's actually mutating the ready lists — and drops it again immediately after. Everything before and after that is either pure register shuffling or touches only the one task's own stack, so it doesn't need protecting.

SVC_Handler

FIRES EXACTLY ONCE · BOOTSTRAPS

PendSV's whole trick depends on there being an *outgoing* task to save. The very first time the scheduler starts, there isn't one — nothing has run under FreeRTOS yet. So starting the scheduler doesn't go through PendSV at all; it goes through SVC, once, to fake that first switch.

The setup for it happens in `privPortStartFirstTask()`, called from `xPortStartScheduler()`:

```

port.c - privPortStartFirstTask (naked)

ldr    r0, =0xE000ED08    ; address of VTOR, the Vector Table Offset Register
ldr    r0, [r0]           ; r0 = the vector table's base address
ldr    r0, [r0]           ; r0 = word[0] of that table = the reset stack top
msr    msp, r0            ; put the Main Stack Pointer back where reset left it

cpsie i                  ; unmask interrupts (clear PRIMASK)

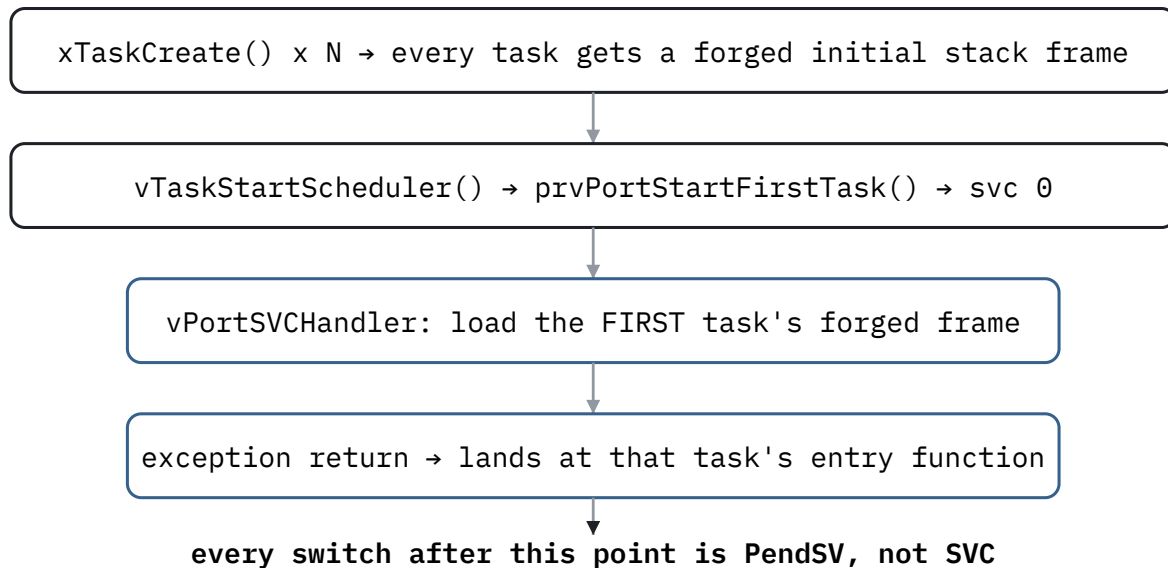
```

```

cpsie f                ; unmask faults      (clear FAULTMASK)
dsb
isb
svc 0                  ; --- trigger the SVC exception ---
nop

```

That single `svc 0` is the only place in the entire codebase this handler is ever invoked from:



port.c – vPortSVCHandler (naked)

```

ldr r3, =pxCurrentTCB ; r3 = &pxCurrentTCB
ldr r1, [r3]           ; r1 = pxCurrentTCB - whichever task the scheduler
                        ; already chose to run first
ldr r0, [r1]           ; r0 = its saved stack pointer - this is the FAKE
                        ; frame pxPortInitialiseStack() built at create-time
ldmia r0!, {r4-r11}    ; pop the fake r4-r11 (dummy values - never real,
                        ; doesn't matter, nothing depends on their contents)
msr psp, r0            ; PSP now points at what's left of that fake frame -
                        ; laid out to look exactly like a hardware exception
isb

mov r0, #0
msr basepri, r0        ; make sure nothing is left masked

orr r14, #0xd          ; force EXC_RETURN to "Thread mode, use PSP"
bx r14                ; exception return - hw pops r0-r3/r12/PC/xPSR from
                        ; the fake frame. PC was set to the task's entry poi
                        ; so execution lands there as if it had always been

```

Notice the shape: it's the second half of `PendSV` , almost verbatim — load stack pointer, `pop r4-r11` , exception return. There's no "save the old context" step, because there is no old context. The whole design of `SVC` here is borrowing `PendSV` 's restore path and skipping its save path, precisely because this only ever needs to run once.

stm32f103 workspace notes · grounded in FreeRTOS-Kernel/portable/GCC/ARM_CM3/port.c